

MT-WAM: Reorienting the One-Pass Predictive Representation Toward Action Generation

Yiguang Yang^{1,2}, Jiankun Peng^{1,2}, Xiaoming Wang^{1,2}, Yiran Zhang^{1,2}, and Zhibo Fang¹

¹Aerospace Information Research Institute, Chinese Academy of Sciences

²University of Chinese Academy of Sciences

Abstract—FAST-WAM shows that video–action co-training improves control without generating future video at inference, making the representation from a single video diffusion Transformer forward central to action generation. However, future-observation prediction does not explicitly prioritize the future dynamics and visual structure needed for control. We present MT-WAM, which retains the original training objectives and adds complementary supervision for future two-dimensional point trajectories and visual features. A lightweight dual-stream branch copied from the video backbone’s final blocks provides target-specific processing, while a structured attention mask prevents cross-stream attention. Motion-stream tokens supply additional dynamics conditions to the action expert. Future visual-feature prediction provides supervision in a feature space that captures object and spatial structure. This supervision trains the video backbone to provide more informative visual context for action generation under changing visual conditions, without adding visual-feature-stream tokens to action conditioning. At inference, MT-WAM uses video and motion caches computed once per replan and skips future-video prediction. Without additional embodied policy pretraining, MT-WAM achieves 98.2% success on LIBERO and 73.7% on LIBERO-PLUS, exceeding FAST-WAM by 23.8 percentage points on the latter. On ROBOTWIN 2.0 Clean2Rand, Random success increases from 6.30% to 19.40%; across four real-world tasks, average success increases from 67.0% to 77.8%. Code repository: <https://github.com/Alexi1984/MT-WAM>.

Index Terms—Robot manipulation, video prediction, vision-language-action models, world action models.

I. INTRODUCTION

ROBOT manipulation requires policies to anticipate how a scene will change during interaction and use that anticipation to generate actions. World action models (WAMs) couple future-observation prediction with action generation, allowing visual and dynamic priors acquired through video pretraining to shape representations for control [1]. These priors help policies model scene evolution, object interactions, and state transitions, and support generalization across tasks and visual conditions. Many WAMs instantiate and denoise future video tokens at inference, using the representations formed during explicit future modeling to condition action generation. FAST-WAM separates the contribution of training-time video prediction from that of inference-time future generation through controlled comparisons [1]. It shows that explicit future generation can be omitted while retaining the control benefits of video–action co-training. The video diffusion Transformer (Video DiT) processes the current observation once and populates its per-layer key–value (KV) caches, which the action expert reuses throughout iterative action denoising. FAST-

WAM achieves performance comparable to explicit future-imagination variants while substantially reducing inference latency. At the same time, its controlled comparisons show that removing video–action co-training causes a large performance drop. Once future generation is skipped, all predictive information from the video model reaches the action expert through this one-pass representation. The representation therefore becomes the critical interface between future-observation prediction and action generation.

The central question is whether this interface makes the information needed for action generation sufficiently prominent. Generating an action requires identifying the relevant objects, anticipating how they will move and interact, and estimating the resulting state transitions. The original future-observation prediction objective is not designed specifically for this purpose. Reconstructing a complete future observation requires the Video DiT hidden states to encode future motion and interaction progress together with the colors, textures, illumination, backgrounds, and other visual details needed to reproduce the future scene. Some of these visual details are not directly relevant to the next action, yet they still shape the same hidden states. The original objective therefore does not prioritize control-relevant predictive information in the representation supplied to the action expert [2]–[5].

Building on this view, we present MT-WAM, which improves the one-pass predictive representation through complementary supervision for future dynamics and visual structure (Fig. 1). It retains video–action co-training and the original future-observation prediction objective while adding future 2D point-trajectory and visual-feature prediction. CoTracker3 [6] provides 2D point-trajectory targets that describe image-space motion and supervise object motion, interaction progress, and state transitions relevant to action generation. Future visual-feature targets extracted by a frozen DINOv2 encoder [7] complement future-observation prediction with semantic information that is relatively robust to low-level appearance variation.

Separate prediction heads leave the preceding Video DiT transformations shared across these heterogeneous targets, whereas a complete Transformer expert for each target duplicates the entire processing path, increasing parameter count and training cost. MT-WAM therefore copies the final M Video DiT blocks to form a dynamic branch (Fig. 2), while the preceding blocks remain shared across the three prediction objectives. Two copies of the reference-frame latents distribute the demands of future-observation prediction, action gener-

MT-WAM Reorienting the one-pass predictive representation toward action generation

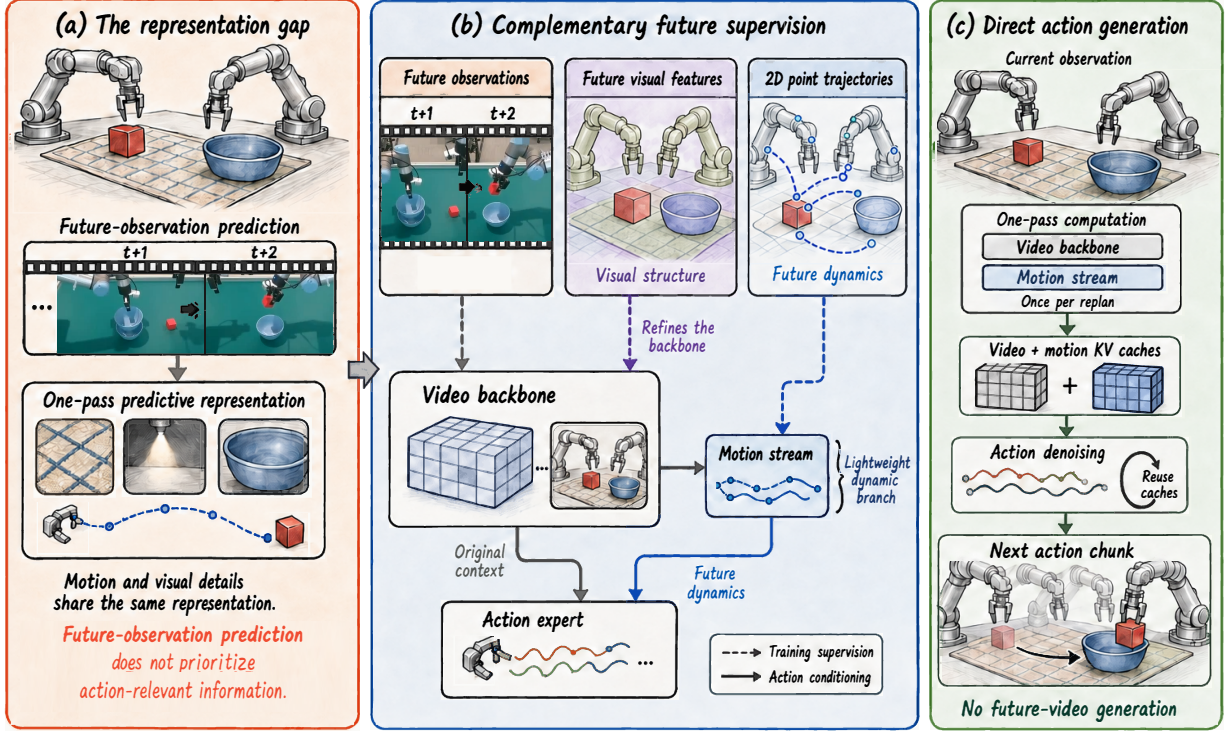


Fig. 1. **MT-WAM overview.** MT-WAM augments future-observation prediction with 2D point-trajectory and future visual-feature targets through a dual-stream dynamic branch. Motion-stream tokens supply additional dynamics conditions, while future visual-feature supervision shapes the scene context that the video backbone provides to the action expert.

ation, and the two additional objectives. Within the branch, the motion and visual-feature streams are initialized from the reference-frame hidden states available to the branch and use different learnable stream embeddings and independently updated stream-specific feed-forward networks (FFNs). A structured attention mask prevents the streams from attending to each other. Action queries attend to the motion-stream tokens, adding future-dynamics information to the action expert’s original conditioning context, but do not attend to the visual-feature-stream tokens. Future visual-feature prediction encourages the Video DiT to capture semantic cues for identifying objects, locating interaction regions, and anticipating scene changes. Learning to predict future scene features thus trains the backbone to provide informative visual context for action generation under varying appearance and scene configurations. The motion and visual-feature prediction heads are used only during training. At inference, MT-WAM skips future video prediction and directly generates actions using the video and motion KV caches computed once per replan.

We evaluated MT-WAM on LIBERO, LIBERO-PLUS, ROBOTWIN 2.0, and four real-world single- and dual-arm manipulation tasks. Without additional embodied policy pre-training, MT-WAM increased the LIBERO-PLUS Overall success rate by 23.8 percentage points, the ROBOTWIN 2.0 Clean2Rand Random success rate by 13.10 points, and the average real-world success rate by 10.8 points relative to FAST-

WAM. The gains are more pronounced under distribution shift and in real-world manipulation than on the standard simulation benchmarks.

Our main contributions are threefold:

- **Action-oriented learning of one-pass predictive representations.** We introduce MT-WAM, which improves the one-pass predictive representation through a joint design of complementary future prediction targets, target-specific processing, and distinct connections to action learning.
- **Lightweight target-specific processing and selective action conditioning.** MT-WAM uses a partially copied dynamic branch, stream-specific feed-forward networks, and a structured attention mask to process heterogeneous future targets. Future visual-feature prediction shapes the visual context produced by the backbone, complementing the dynamics conditions supplied directly by the motion stream. Together, these pathways connect predictive learning to both the visual and dynamic information used for action generation. MT-WAM retains the direct action-generation interface at inference.
- **Evaluation across simulation, out-of-distribution (OOD) settings, and real-world manipulation.** Without additional embodied policy pretraining, MT-WAM achieves a 98.2% success rate on LIBERO and improves over FAST-WAM by 23.8, 13.10, and 10.8 per-

centage points on LIBERO-PLUS, the ROBOTWIN 2.0 Clean2Rand Random condition, and four real-world tasks, respectively. Controlled ablations support the asymmetric roles of the two branch streams.

II. RELATED WORK

A. Vision-Language-Action Policies

Vision-language-action (VLA) policies map visual observations, language instructions, and robot state to actions by combining pretrained backbones with robot demonstrations [8]–[13]. Action chunking and diffusion- or flow-based heads support direct action generation [14]–[16]. MT-WAM retains this interface but trains its video backbone with future-observation prediction and two complementary predictive targets.

B. World Action Models

World action models (WAMs) couple future-observation prediction with action generation so that video priors shape representations for control [1]. Many explicitly generate future visual trajectories or jointly denoise future video and actions. FAST-WAM retains video prediction during training but generates actions from the video KV cache produced by a single backbone forward [1]. ImageWAM similarly conditions action generation on KV caches produced by an image-editing backbone [17]. MT-WAM retains the one-pass interface of FAST-WAM and focuses on improving the representation produced by that pass.

C. Beyond RGB Predictive Supervision in VLA and World Models

VLA policies use auxiliary prediction and feature distillation to inform action generation. DreamVLA predicts future dynamic-region content, depth, and semantics with separate queries [18]. FoMoVLA trains foresight queries through future-feature prediction and uses their hidden states to condition 2D point-trajectory prediction and action generation [19]. Track4Action distills world-centric 3D tracker features from action-aligned clips into queries used by its action head [20].

In WAMs, AGRA aligns video hidden states used for action conditioning with pretrained visual features [21], GeoSemWAM adds dense prediction heads for future geometry and semantic segmentation [22], and EgoWAM compares alternative pixel, visual-feature, and camera-stabilized 3D-motion targets for human–robot co-training [23].

JOPAT jointly denoises future visual latents, 2D point tracks, and actions while predicting track visibility [24], whereas FlowWAM generates optical-flow videos as an action representation [25]. For target-specific processing, X-WAM copies Video DiT tail blocks for depth prediction [26], and MECoWAM adds a separate training-time 4D expert with temporary current-frame geometry conditioning [27]. DreamWAM jointly denoises RGB and optical-flow latents, while geometry and semantic supervision refine video hidden states through gated residual branches [28].

Together, these studies highlight target-specific processing and action conditioning alongside the choice of predictive

targets. MT-WAM combines a partially copied Video DiT tail with asymmetric action conditioning while skipping future prediction at inference. Motion-stream tokens supplement the action expert’s original conditioning inputs, whereas visual-feature supervision refines the shared video backbone without adding visual-feature-stream tokens to action conditioning.

III. METHOD

A. Problem Formulation

We consider a WAM with a direct-policy interface [1]. At each decision step, the policy receives a current multi-view RGB observation I_0 , a proprioceptive state q_0 , and a language instruction l , and predicts an action chunk $a_{1:H}$ of horizon H . At inference, the video backbone processes I_0 once to compute the video key–value (KV) cache. Together with q_0 and l , this cache forms the action expert’s original conditioning inputs, denoted by $\mathcal{C}_{\text{base}}$:

$$p_{\theta}(a_{1:H} \mid I_0, q_0, l) = p_{\theta_a}(a_{1:H} \mid \mathcal{C}_{\text{base}}). \quad (1)$$

During training, the future-observation prediction loss updates the video backbone and shapes the hidden states from which the video KV cache is computed.

MT-WAM retains the future-observation prediction objective and adds future 2D point-trajectory and visual-feature prediction during training. The corresponding motion and visual-feature streams maintain separate hidden states and serve different downstream roles. At inference, the per-layer K/V projections of the motion-stream tokens form the motion KV cache $\mathcal{C}_m$, which augments the action expert’s original conditioning inputs:

$$p_{\theta}(a_{1:H} \mid I_0, q_0, l) = p_{\theta_a}(a_{1:H} \mid \mathcal{C}_{\text{base}}, \mathcal{C}_m). \quad (2)$$

No visual-feature KV cache is included in the action expert’s conditioning inputs. The visual-feature prediction loss instead trains the visual-feature stream and the shared video backbone.

B. Dual-Stream Dynamic Branch

Future video latents, 2D point trajectories, and visual features differ in target space and tensor structure. Target-specific prediction heads separate only their output mappings, whereas assigning a full Transformer expert to each target would duplicate the entire processing path. MT-WAM instead copies the final M blocks of the video backbone to form a dynamic branch (Fig. 2), in which the motion and visual-feature streams receive target-specific processing through stream-specific FFNs. The original video path retains future-observation prediction. Section III-D defines the attention relations among the token groups.

1) *Tail Attachment and Branch Input*: Let the video backbone contain N blocks, with the first $N - M$ shared by the original video path and the dynamic branch. The branch consists of equal-width copies of the final M Video DiT blocks, initialized from the corresponding video blocks and updated independently. It predicts future 2D point trajectories and visual features from the reference-frame hidden states produced by block $N - M$, without receiving noised future video tokens. This confines the additional target-specific processing to the copied tail.

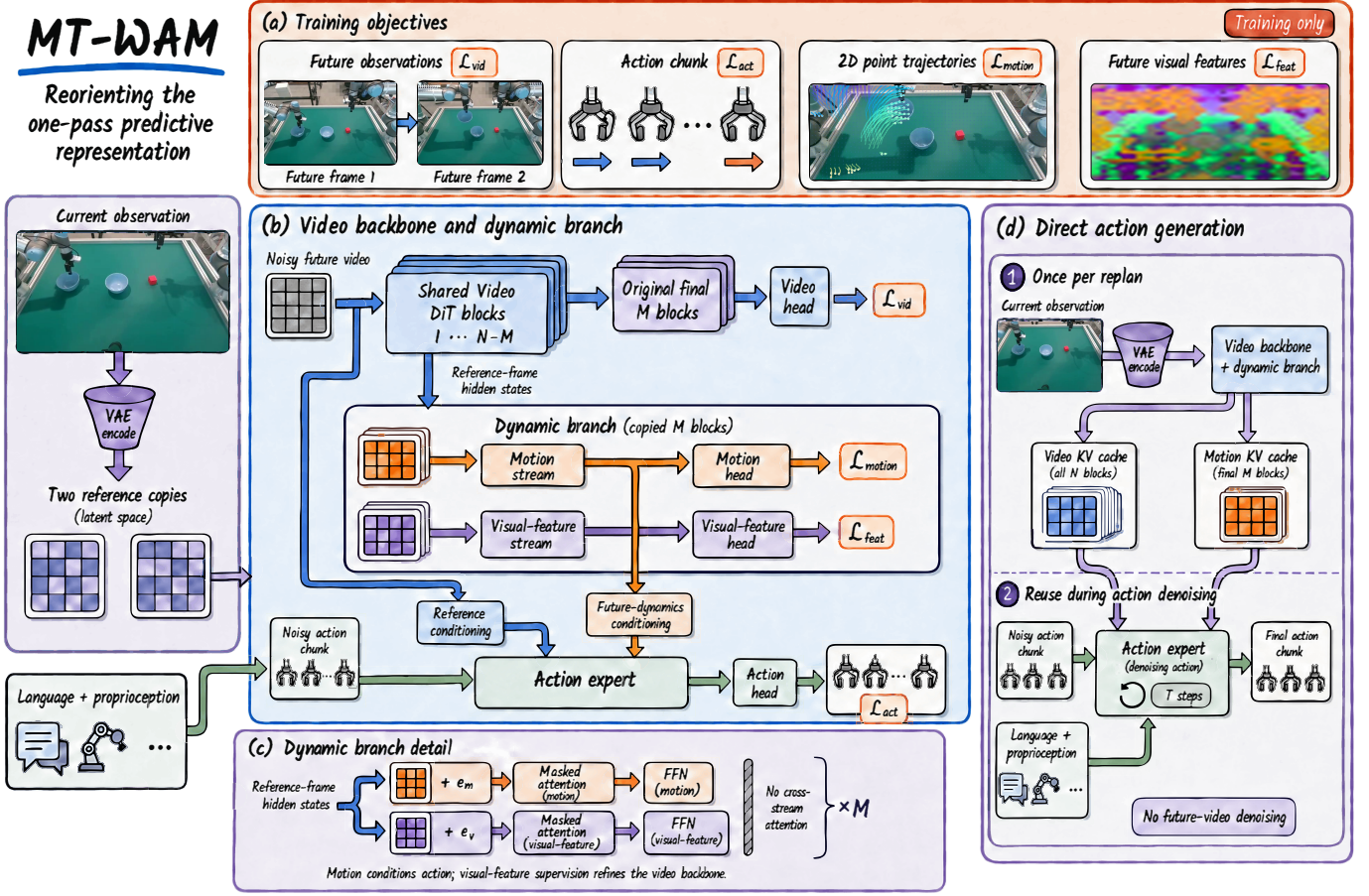


Fig. 2. **MT-WAM training, architecture, and inference.** (a) Training uses future-observation, action, 2D point-trajectory, and future visual-feature objectives. (b) The video path retains future-observation prediction, the action expert generates actions, and an M -block dynamic branch predicts the two added targets. (c) The streams use separate FFNs and a structured attention mask; action queries attend to motion-stream tokens but not visual-feature-stream tokens. (d) The video backbone and dynamic branch populate the video and motion KV caches once per replan, and the action expert reuses both throughout denoising. The future-video, motion, and visual-feature heads are training-only; Fig. 3 shows the complete masks.

2) *Two Reference Copies*: MT-WAM creates two copies of the reference-frame latent tokens to distribute the demands of action conditioning and the two additional prediction objectives. The copies are content-identical before positional encoding: action queries attend to one, while the other provides the reference-frame hidden states used by the motion and visual-feature streams. Future video tokens attend to both copies, allowing the future-observation loss to update the video backbone through both. During training, the dynamic branch therefore predicts 2D point trajectories and future visual features from current-observation hidden states already shaped by future-observation supervision.

At the branch entrance, the reference-frame hidden states associated with the second copy are duplicated to initialize both streams from identical values. Different learnable stream embeddings are then added, allowing the copied Video DiT blocks to distinguish the two predictive roles.

3) *Stream-Specific Feed-Forward Networks*: Sharing one feed-forward network would require the same nonlinear transformation parameters to serve both target spaces. In every copied block, MT-WAM instead duplicates the original feed-forward network into motion and visual-feature FFNs. The

two FFNs start from the same parameters but are updated independently, and each stream always uses its corresponding FFN. This fixed assignment follows modality-specific expert designs [29] and provides separate FFN parameters without a learned router, Top- k selection, or a load-balancing objective. The two streams therefore apply distinct nonlinear transformations throughout the copied tail rather than differing only at their prediction heads.

C. Motion and Visual-Feature Supervision

The motion and visual-feature objectives provide complementary supervision for future dynamics and visual structure. Both losses update the dynamic branch and preceding Video DiT blocks.

1) *Motion Target*: A frozen CoTracker3 tracker [6] tracks G_m points placed at uniform grid-cell centers in the reference frame f_0 of I_0 , across P representative future frames spanning the action horizon and including its endpoint. The cumulative displacement target is

$$\Phi_{bcki} = \frac{\tau_{bci}(f_k) - \tau_{bci}(f_0)}{\gamma} \in \mathbb{R}^2, \quad k = 1, \dots, P, \quad (3)$$

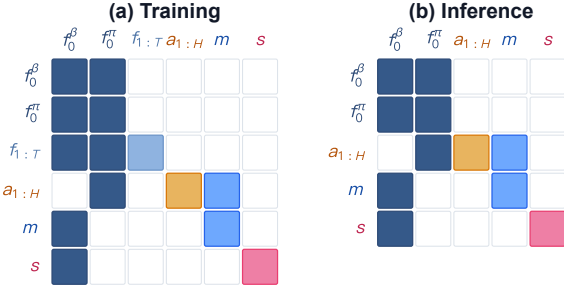


Fig. 3. **Structured attention masks used by MT-WAM.** Rows are queries and columns are keys; colored cells denote allowed attention. f_0^π and f_0^β are the reference copies available to action queries and the two branch streams; $f_{1:T}$, $a_{1:H}$, m , and s denote the future video, action, motion, and visual-feature groups. Inference omits $f_{1:T}$.

where $i = 1, \dots, G_m$, $\tau_{bci}(f_k)$ denotes pixel coordinates, and γ is the spatial compression factor of the variational autoencoder (VAE). Indices b and c identify the B samples and C camera views. Binary masks m_{bk} and v_{bcki} exclude padded frames and invisible points, respectively. With $w_{bcki} = m_{bk}v_{bcki}$, the prediction loss is normalized within each sample-camera pair:

$$\mathcal{L}_{\text{motion}} = \frac{1}{BC} \sum_{b,c} \frac{\sum_{k,i} w_{bcki} \frac{1}{2} \|\hat{\Phi}_{bcki} - \Phi_{bcki}\|_2^2}{\max(1, \sum_{k,i} w_{bcki})}. \quad (4)$$

All displacements are measured from f_0 ; visible static points remain valid targets with near-zero displacement.

2) *Visual-Feature Target*: A frozen DINOv2 ViT-B/14 encoder [7] extracts G_s patch targets $\psi_{bckj} \in \mathbb{R}^{d_\psi}$ per future frame and camera, where j indexes patches. Only the P future frames are used, excluding f_0 . These targets complement image-space motion supervision with object-, part-, and region-level visual structure. Regressing them encourages the video backbone to retain this structure under low-level appearance variation. Predictions $\hat{\psi}_{bckj}$ are optimized with a cosine loss using the same frame-validity mask and per-pair normalization:

$$\mathcal{L}_{\text{feat}} = \frac{1}{BC} \sum_{b,c} \frac{\sum_{k,j} m_{bk} (1 - \cos(\hat{\psi}_{bckj}, \psi_{bckj}))}{\max(1, \sum_{k,j} m_{bk})}. \quad (5)$$

3) *Prediction Heads*: Each stream uses a separately parameterized mask-token decoder [18], [19]. Each decoder linearly projects its stream’s final hidden states and concatenates them with mask tokens carrying fixed 2D sine-cosine positional encodings and learned future-frame embeddings. A shallow Transformer followed by a linear layer maps the decoded mask tokens to a $P \times G_m \times 2$ displacement tensor or a $P \times G_s \times d_\psi$ feature tensor. Both heads are omitted at inference.

D. Structured Attention Mask

The structured attention mask extends the video-action mask of the direct-policy interface to the two reference copies and the dynamic branch (Fig. 3) [1]. The two reference copies attend to each other but not to noised future video inputs.

Future-video attention is restricted to the future sequence and both reference copies. Each branch stream attends within its own sequence and to the reference copy available to both streams, preventing direct mixing of their target-specific hidden states.

Within the action sequence, attention remains bidirectional. Action queries also attend to their reference copy and motion-stream tokens, incorporating future-dynamics information about object motion, interaction progress, and state transitions into the original conditioning context. The motion stream is therefore trained by both the motion and action losses, linking future-dynamics learning to the requirements of action generation.

The visual-feature loss instead updates the visual-feature stream and propagates through its reference-frame inputs to the shared video backbone. Attention between the two reference copies allows this supervision to update backbone parameters that also produce the reference-frame hidden states used for action conditioning. Through this path, future visual-feature prediction supervises the backbone computations that produce the reference-frame keys and values read by the action expert, linking future scene modeling to the visual conditions for action generation (Fig. 2(c) and (d)).

E. Training Objective

The full objective is

$$\mathcal{L} = \lambda_{\text{vid}} \mathcal{L}_{\text{vid}} + \lambda_{\text{act}} \mathcal{L}_{\text{act}} + \lambda_{\text{motion}} \mathcal{L}_{\text{motion}} + \lambda_{\text{feat}} \mathcal{L}_{\text{feat}}. \quad (6)$$

The first two terms are the original video and action flow-matching objectives [1], [30], [31]. They act on future video latents and action chunks, respectively, with independently sampled flow times. The motion and visual-feature prediction-head outputs are compared with targets extracted from future observations. Unlike the flow-matching terms, these additional losses use neither noisy interpolation nor flow-time weighting. The video backbone, action expert, and dynamic branch are optimized jointly, while the pretrained VAE and text encoder remain frozen. Appendix A reports the loss weights.

F. Inference

At inference (Fig. 2(d)), MT-WAM omits the motion and visual-feature prediction heads and skips future video prediction. The video backbone processes both reference copies in a single pass, after which the dynamic branch computes the motion-stream hidden states from the reference-frame hidden states available to both branch streams. These computations populate the per-layer video and motion KV caches before action denoising. At every denoising step, the action expert reuses both caches according to the structured attention mask. Because the training mask already prevents action queries from attending to noised future video tokens, removing these tokens at inference does not change the information available for action generation (Fig. 3). The video backbone and dynamic branch are each evaluated once per replan, and MT-WAM directly generates actions without explicitly generating future observations.

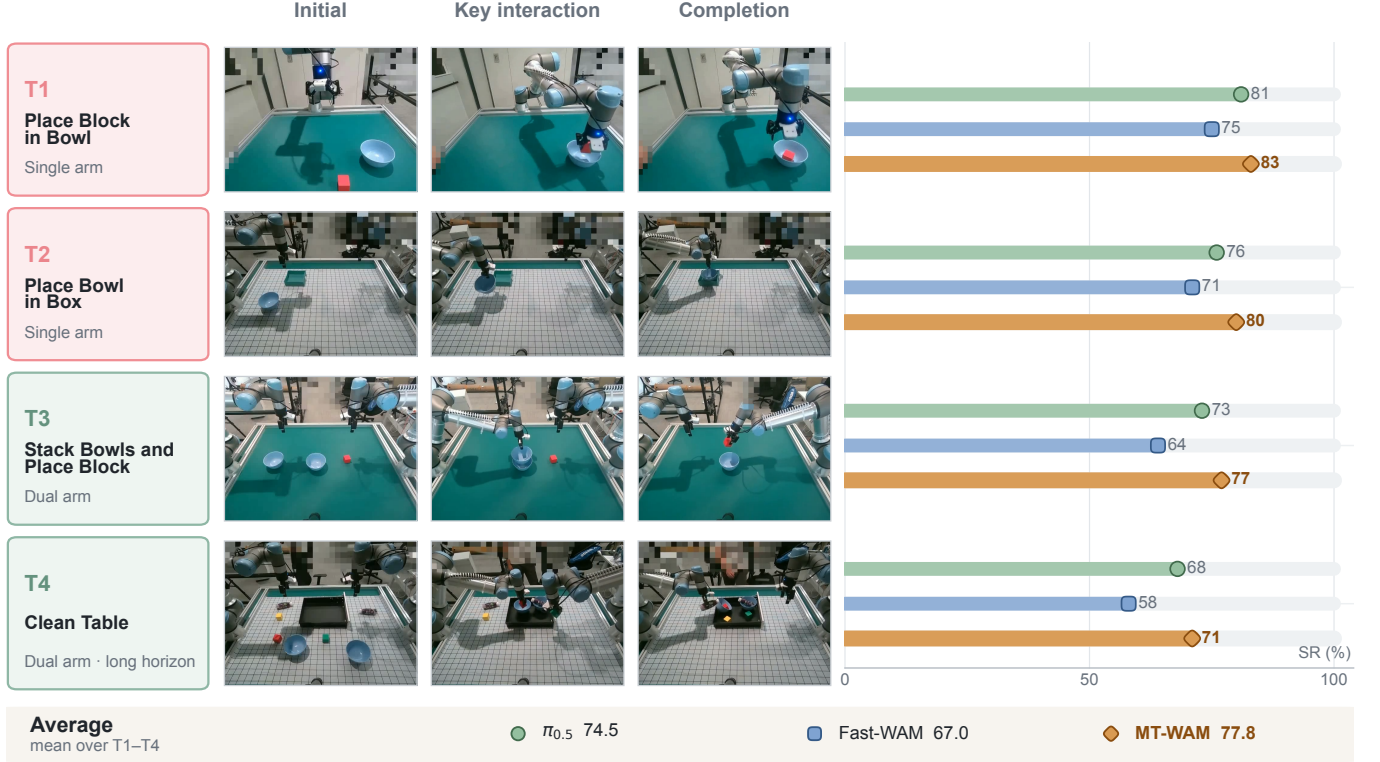


Fig. 4. **Real-world tasks and success rates (%)**. Representative initial, key-interaction, and completion frames for T1–T4 are aligned with task-level SRs. All methods use the same 400 demonstrations and 10-epoch training budget, and are evaluated over the same initial configurations with 100 trials per task; Average is the arithmetic mean over T1–T4.

IV. EXPERIMENTS

A. Experiment Setup

We evaluated MT-WAM on LIBERO, LIBERO-PLUS, and ROBOTWIN 2.0 Clean2Rand (C2R), as well as on four real-world single- and dual-arm manipulation tasks, using task success rate (SR) as the primary metric. In Tables I–IV, PT denotes additional embodied policy pretraining beyond benchmark-specific training. Model initialization, training configurations, and training cost are detailed in Appendix A.

1) *LIBERO and LIBERO-Plus*: We evaluated all methods on LIBERO [32] and LIBERO-PLUS [33] following their standard protocols. Benchmark-specific training used LIBERO demonstrations without augmented LIBERO-PLUS data. We report the four-suite Average on LIBERO and the Overall SR across all 10,030 LIBERO-PLUS variants.

2) *RoboTwin 2.0*: ROBOTWIN 2.0 is a large-scale simulated benchmark for bimanual robot manipulation [34]. We used all 50 Aloha AgileX tasks, which require coordinated dual-arm control across diverse object layouts and scene conditions. In the multi-task setting, we trained one policy per method for 5 epochs on demonstrations from all tasks, using both Clean and Rand demonstrations. We evaluated each method under both conditions over 100 trials per task and report the average SR across the 50 tasks for each condition, together with their arithmetic mean.

3) *RoboTwin 2.0 Clean2Rand*: C2R evaluates generalization to Random environments without Random action supervi-

sion [35]. VLA methods follow a single-stage protocol and are fine-tuned on 50 Clean paired demonstrations per task. WAMs follow a two-stage protocol. Stage 1 pretrains each WAM’s video component for 5 epochs on video sequences from 50 Clean and 500 Random demonstrations per task; for MT-WAM, this stage also trains the dynamic branch. The resulting weights initialize the complete policy, which is then jointly fine-tuned for 5 epochs using only 50 Clean paired demonstrations per task. Random demonstrations therefore provide video-based predictive supervision but no action supervision. Each method is evaluated for 100 Clean and 100 Random rollouts on each of the 50 tasks, totaling 10,000 rollouts per method. We report Clean and Random SR, the absolute Clean-to-Random drop in percentage points, and the descriptive Random retention $100 \text{ SR}_{\text{Random}} / \text{SR}_{\text{Clean}}$.

TABLE I
SUCCESS RATES (%) ON LIBERO

Method	PT	Spatial	Object	Goal	Long	Average
OpenVLA [13]	✓	84.7	88.4	79.2	53.7	76.5
π_0 [16]	✓	96.8	98.8	95.8	85.2	94.1
$\pi_{0.5}$ [36]	✓	98.8	98.2	98.0	92.4	96.9
LingBot-VA [37]	✓	98.5	99.6	97.2	98.5	98.5
Motus [38]	✓	96.8	<u>99.8</u>	96.6	<u>97.6</u>	97.7
FAST-WAM [1]	✗	98.2	100.0	97.0	95.2	97.6
MT-WAM	✗	99.3	99.2	98.3	96.0	<u>98.2</u>

TABLE II
SUCCESS RATES (%) ON LIBERO-PLUS

Method	PT	Cam.	Rob.	Lang.	Light	Bg.	Noise	Lay.	Ovr.
UniVLA [39]	✓	1.80	46.20	69.60	69.00	81.00	21.20	31.90	42.87
OpenVLA-OFT [40]	✓	56.40	31.90	79.50	88.70	93.30	75.80	74.20	69.59
π_0 [16]	✓	13.80	6.00	58.80	85.00	81.40	79.00	68.90	53.63
π_0 -FAST [41]	✓	65.10	21.60	61.00	73.20	73.20	74.40	68.80	61.59
WorldVLA [42]	✓	0.10	27.90	41.60	43.70	17.10	10.90	38.00	25.03
FAST-WAM [1]	✗	16.26	44.13	66.82	79.77	52.60	38.54	61.38	49.86
Fast-WAM-Joint [1]	✗	39.59	60.90	<u>92.32</u>	94.57	57.62	58.59	<u>80.52</u>	68.41
SG-WAM [43]	✗	<u>58.60</u>	48.90	81.40	89.80	<u>86.10</u>	80.70	74.20	<u>73.00</u>
ST-WAM [44]	✗	55.40	60.10	79.30	93.00	74.20	<u>79.50</u>	74.30	72.80
4D-WAM [45]	✗	45.15	64.26	90.63	<u>94.29</u>	57.71	69.08	79.21	71.01
MT-WAM	✗	52.70	<u>61.10</u>	94.70	93.60	64.80	71.30	81.00	73.66

4) *Real-World Experiments*: We evaluated four tasks using UR5 manipulators in single-arm and dual-arm setups, with all visual observations captured by Intel RealSense D405 cameras (Fig. 4). Place Block in Bowl (T1) and Place Bowl in Box (T2) use one arm; Stack Bowls and Place Block (T3) and the long-horizon Clean Table task (T4) use two arms. Each task contained 100 demonstrations, evenly divided between solid and patterned tabletop backgrounds. We pooled all 400 demonstrations across tasks, camera viewpoints, and background types to train one multi-task policy per method for 10 epochs. Each method was evaluated over 100 trials per task, with a solid or patterned tabletop background selected randomly for every trial and the same initial configurations used across methods. We report task-level SR and the arithmetic mean across the four tasks. Appendix B provides additional details.

B. Main Results

1) *LIBERO and LIBERO-Plus*: On LIBERO, MT-WAM achieved an average SR of 98.2%, compared with 97.6% for FAST-WAM (+0.6 percentage points; Table I). It improved LIBERO-Spatial, LIBERO-Goal, and LIBERO-Long by 1.1, 1.3, and 0.8 percentage points, respectively, while scoring 0.8 points lower on LIBERO-Object. These small suite-level differences indicate comparable performance on the standard LIBERO tasks.

On LIBERO-PLUS, MT-WAM achieved an Overall SR of 73.66% without additional embodied policy pretraining, the highest among the methods compared in Table II and 23.80 percentage points above FAST-WAM. The gains extended across all seven perturbation axes, with the largest improvements under Camera, Noise, Language, and Layout, where SR increased by 36.44, 32.76, 27.88, and 19.62 percentage points, respectively. These results, together with the comparable performance on standard LIBERO, show that MT-WAM maintains performance on the original tasks and improves robustness when observations, instructions, or scene conditions change. These findings support the central premise of MT-WAM: improving the one-pass predictive representation along future-dynamics and visual-structure dimensions can strengthen action generation without explicitly predicting future video at inference.

TABLE III
SUCCESS RATES (%) ON ROBOTWIN 2.0 CLEAN/RAND

Method	PT	Clean	Rand	Average
π_0 [16]	✓	65.92	58.40	62.16
$\pi_{0.5}$ [36]	✓	82.74	76.76	79.75
ABot-M0 [46]	✗	81.20	80.40	80.80
Motus [38]	✓	88.66	87.02	87.84
Motus from Wan 2.2 [38]	✗	77.56	77.00	77.28
LingBot-VA [37]	✓	<u>92.90</u>	91.50	<u>92.20</u>
FAST-WAM [1]	✗	91.88	<u>91.78</u>	91.83
MT-WAM	✗	93.18	93.26	93.22

2) *RoboTwin 2.0 and Clean2Rand*: In the ROBOTWIN 2.0 Clean/Rand setting, MT-WAM achieved an average SR of 93.22% without additional embodied policy pretraining, exceeding FAST-WAM by 1.39 percentage points (Table III). It achieved the highest SR among the compared methods in both conditions, with similar performance on Clean and Rand, showing consistent control across conditions included in action-supervised training.

ROBOTWIN 2.0 Clean2Rand instead evaluates generalization to Random environments without Random action supervision. MT-WAM achieved a Random SR of 19.40%, exceeding FAST-WAM by 13.10 percentage points (Table IV). HALO achieved a higher Clean SR, while HALO and BagelVLA achieved higher Random SRs; both methods use additional embodied policy pretraining. The larger advantage over FAST-WAM on Random than on Clean supports the central premise of MT-WAM: improving the one-pass predictive representation through complementary future supervision and target-specific processing can strengthen action generation beyond Clean action supervision.

3) *Real-World Experiments*: Fig. 4 aligns representative task progressions with their task-level SRs. Without additional embodied policy pretraining, MT-WAM achieved an average SR of 77.8%, compared with 74.5% for $\pi_{0.5}$, which uses such pretraining, and 67.0% for FAST-WAM [1]. Among the compared methods, MT-WAM achieved the highest SR on all four tasks. These tasks span single-arm object placement, multi-step bimanual manipulation, and long-horizon multi-object manipulation. Relative to FAST-WAM, MT-WAM improved SR by 8, 9, 13, and 13 percentage points on T1 (Place Block in

TABLE IV
SUCCESS RATES (%) ON ROBOTWIN 2.0 CLEAN2RAND

Method	PT	Clean	Random	Absolute drop (pp)	Random retention (%)
π_0 [16]	✓	46.40	16.30	30.10	35.1
RDT [47]	✓	34.50	13.70	20.80	39.7
UP-VLA [48]	✓	52.90	15.20	37.70	28.7
Diffusion Policy (DP) [15]	✗	28.00	0.60	27.40	2.1
HALO [49]	✓	80.50	26.40	54.10	32.8
BagelVLA [50]	✓	75.30	<u>20.50</u>	54.80	27.2
FAST-WAM [1]	✗	71.90	6.30	65.60	8.8
StarVLA [51]	✗	65.72	11.68	54.04	17.8
MT-WAM	✗	<u>75.56</u>	19.40	56.16	25.7

Bowl), T2 (Place Bowl in Box), T3 (Stack Bowls and Place Block), and T4 (Clean Table), respectively. The gains were largest on T3 and T4, which place greater demands on maintaining interaction progress and state-transition information across successive action chunks. Each trial used a randomly selected solid or patterned tabletop background, and the gains persisted across this background variation. Both FAST-WAM and MT-WAM skip explicit future generation at inference, so the improvement does not depend on an additional future video rollout. These results show that complementary future-motion and visual-feature supervision strengthens real-world control while preserving direct action generation.

4) *Inference Efficiency*: Table V compares inference efficiency on LIBERO-PLUS. All models were measured under the same inference configuration on a single NVIDIA RTX PRO 6000 96 GB GPU. We report the number of floating-point operations in trillions (TFLOPs).

TABLE V
INFERENCE EFFICIENCY ON LIBERO-PLUS USING ONE NVIDIA RTX PRO 6000 96 GB GPU

Method	Future video	Overall SR (%)	Latency (ms)	TFLOPs
FAST-WAM [1]	✗	49.86	296.325	3.736949
Fast-WAM-Joint [1]	✓	68.41	503.729	30.672534
MT-WAM	✗	73.66	313.480	5.142984

MT-WAM achieves the highest Overall SR among the three methods in Table V. Compared with FAST-WAM, it improves Overall SR by 23.80 percentage points, with increases of 5.79% in latency and 37.63% in FLOPs. Thus, the additional computation is accompanied by a substantial improvement in control performance, while inference latency remains close to FAST-WAM.

Compared with Fast-WAM-Joint, MT-WAM uses approximately one-sixth of the floating-point operations while achieving an Overall SR 5.25 percentage points higher. Our design improves the one-pass predictive representation, enabling MT-WAM to achieve a higher Overall SR than Fast-WAM-Joint without restoring full future-video denoising at inference.

C. Ablation Study

Our ablations address three questions about the design of MT-WAM: whether action queries should attend to the visual-feature-stream tokens, whether the visual-feature stream

contributes to control performance, and how performance varies with the branch depth M . Fig. 5 summarizes the first two comparisons, and Fig. 6 reports the branch-depth study. Each comparison changes the design factor specified by its corresponding question and follows the LIBERO-PLUS protocol in Section IV-A.

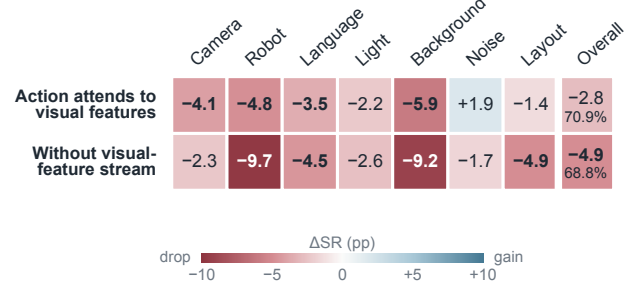


Fig. 5. LIBERO-PLUS ablation effects relative to MT-WAM. Each cell reports the change in SR in percentage points; the Overall SR entries additionally report the absolute SR. MT-WAM achieves an Overall SR of 73.7%.

1) *Should Action Queries Attend to the Visual-Feature Stream*: The controlled comparison changes only one attention rule: action queries attend to the visual-feature-stream tokens as well as the motion-stream tokens, while the two streams, prediction objectives, and parameter count remain unchanged. The Overall SR falls from 73.7% to 70.9%, a reduction of 2.8 percentage points, and the SR decreases on six of the seven perturbation axes. The largest drops occur under Background (−5.9), Robot (−4.8), and Camera (−4.1). Noise is the only exception, with a 1.9-point gain. Because no other architectural component changes, the comparison isolates the effect of adding visual-feature-stream tokens to the action-conditioning context. The result shows that this additional visual-feature conditioning does not improve action generation and supports excluding it.

2) *Does the Visual-Feature Stream Contribute*: Removing the visual-feature stream lowers the Overall SR from 73.7% to 68.8%, a reduction of 4.9 percentage points, and decreases the SR on all seven perturbation axes. The largest reductions occur under Robot (−9.7), Background (−9.2), and Layout (−4.9). The reductions span changes in robot initial states, object layouts, language instructions, lighting, backgrounds, and image noise. The visual-feature stream therefore contributes across both structural and visual perturbations.

Taken together, the two ablations support the intended asymmetric roles of the branch streams. The visual-feature stream improves control through its training objective and target-specific processing, but allowing action queries to attend to its tokens reduces the Overall SR. The visual-feature stream therefore refines the video backbone during training, while the motion stream supplies the additional future-dynamics information used directly for action generation.

3) *How Does Performance Vary with Branch Depth?*: We vary M while keeping all other settings fixed (Fig. 6).

Increasing M starts the dynamic branch at an earlier Video DiT block. It allocates more copied blocks and parameters to

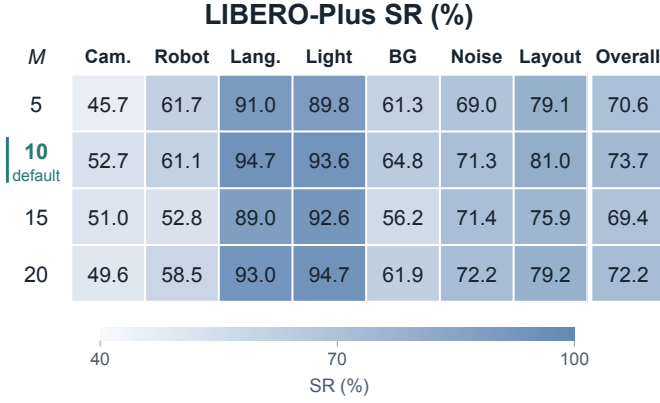


Fig. 6. Success rates (%) across branch depths on LIBERO-PLUS.

target-specific processing, but leaves fewer video blocks shared before the branch. Among the evaluated depths, $M = 10$ achieves the highest Overall SR at 73.7%, although no single configuration leads every perturbation axis. The axis-specific leaders are distributed across the settings: $M = 5$ performs best under Robot and $M = 20$ under Light and Noise, whereas $M = 10$ leads Camera, Language, Background, and Layout. The Overall SR advantage of $M = 10$ therefore reflects a balanced profile rather than a peak on one axis. It also uses half as many copied blocks as $M = 20$ while achieving an Overall SR 1.5 percentage points higher. Performance is thus non-monotonic with branch depth. We use $M = 10$ as the default because it attains the highest Overall SR among the evaluated depths with half as many copied blocks as $M = 20$.

V. DISCUSSION AND CONCLUSION

We introduced MT-WAM, which improves the one-pass predictive representation along future-dynamics and visual-structure dimensions through a lightweight dynamic branch. Gains over FAST-WAM [1] are more pronounced under distribution shift and in real-world manipulation: 23.80 percentage points on LIBERO-PLUS, 13.10 points in the ROBOTWIN 2.0 Clean2Rand Random condition, and 10.8 points on average across real-world tasks. The largest real-world gains occur on tasks requiring multi-step object interactions, consistent with the importance of interaction progress and state-transition information.

The ablations support the asymmetric roles of the two streams. The visual-feature pathway contributes through training the backbone’s scene encoding, while motion-stream tokens provide the additional dynamics conditions used directly for action generation. Removing the visual-feature stream lowers LIBERO-PLUS Overall SR by 4.9 percentage points, whereas allowing action queries to attend to its tokens lowers it by 2.8 points. These results indicate that a predictive stream can improve control through its contribution to training without its tokens serving as additional action conditions.

The dynamic branch does more than add capacity: its starting point sets the boundary between shared video representation learning and target-specific processing. The non-monotonic depth results suggest that control benefits from

balancing shared predictive features with independent transformations for heterogeneous targets. A copied tail provides these target-specific transformations without duplicating the full video-processing hierarchy for each target, limiting the additional computation required for specialization. The efficiency results support the practicality of this allocation: MT-WAM strengthens action generation with a modest latency increase over FAST-WAM and lower computation than joint video-action denoising.

Limitations. The 2D point-trajectory target describes image-space motion but does not directly encode metric 3D geometry or contact states. Adding these targets could extend the dynamic branch’s predictive information. Evaluation covers one pretrained video backbone and UR5 single- and dual-arm configurations, without additional embodied policy pretraining. Further evaluation across backbones, embodiments, and longer-horizon tasks would examine the scope of the design, while large-scale embodied policy pretraining would test its complementarity with data scaling.

Overall, MT-WAM shows that, for WAMs that omit explicit future generation, translating predictive supervision into control gains depends on making the one-pass predictive representation better serve action generation. Results from simulation, real-world experiments, controlled ablations, and efficiency analysis jointly show that complementary supervision for future dynamics and visual structure, together with the asymmetric design, strengthens direct action generation without restoring iterative future-video denoising.

APPENDIX A TRAINING DETAILS

The video backbone is initialized from the Wan 2.2-TI2V-5B checkpoint [52].¹ The action expert uses the Action DiT architecture and is also initialized from the Wan 2.2 Video DiT. Exact-shape backbone tensors are copied directly, while size-mismatched tensors are linearly interpolated with width-dependent scaling. The action encoder and output head are randomly initialized.

All models are trained on 8 NVIDIA A100 GPUs with DeepSpeed ZeRO-1 and bfloat16 precision. We use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, learning rate 1×10^{-4} , and weight decay 1×10^{-2} . The learning rate follows a warmup-cosine schedule: linear warmup spans the first $0.05T_{\text{total}}$ steps, and the minimum learning rate is 0.01 times the initial rate. Gradients are clipped to a global norm of 1.0. The loss weights are $\lambda_{\text{vid}} = \lambda_{\text{act}} = 1.0$, $\lambda_{\text{motion}} = 0.5$, and $\lambda_{\text{feat}} = 0.25$. Table VI summarizes the dataset-specific configurations.

The C2R protocol has two stages. Stage 1 uses video sequences from 50 Clean and 500 Random demonstrations per task to pretrain the video backbone and dynamic branch for 5 epochs. Stage 2 starts from this checkpoint and jointly fine-tunes the complete policy for 5 epochs using only 50 Clean paired demonstrations per task. Random demonstrations therefore provide video-based predictive supervision in Stage 1 but no action supervision. Table VII reports the training cost and global batch size for each protocol.

¹<https://huggingface.co/Wan-AI/Wan2.2-TI2V-5B>

APPENDIX B

REAL-WORLD EXPERIMENT DETAILS

We used UR5 manipulators in single-arm and dual-arm setups, with all visual observations captured by Intel RealSense D405 cameras. Place Block in Bowl and Place Bowl in Box used one arm; Stack Bowls and Place Block and the long-horizon Clean Table task used two arms. We collected 100 demonstrations per task, evenly divided between solid and patterned tabletop backgrounds. The resulting 400 demonstrations spanned multiple camera viewpoints and were used to train one multi-task policy per method for 10 epochs. Training took approximately 17 hours on 8 NVIDIA A100 GPUs with a global batch size of 48. Each method was evaluated over 100 trials per task. Every trial used a randomly selected solid or patterned background, and all methods shared the same initial configurations. We report task-level SR and the arithmetic mean over the four tasks.

TABLE VI
DATASET-SPECIFIC TRAINING CONFIGURATIONS

Configuration	LIBERO	ROBOTWIN 2.0	
		Clean/Rand	C2R
Camera views	2	3	3
Camera layout	Horizontal	Wrist-horizontal + vertical	Wrist-horizontal + vertical
Input resolution	224×448	384×320	384×320
Observation window	33	33	33
Action chunk length	32	32	32
Action dimension	7	14	14
Training epochs	10	5	Stage 1: 5 Stage 2: 5

TABLE VII
TRAINING COST AND GLOBAL BATCH SIZE

Protocol	Time	Batch size
LIBERO	16 h	96
ROBOTWIN 2.0	4 days	48
ROBOTWIN 2.0 C2R Stage 1	4 days	48
ROBOTWIN 2.0 C2R Stage 2	18 h	48
Real-world multi-task	17 h	48

REFERENCES

- [1] T. Yuan *et al.*, “Fast-WAM: Do world action models need test-time future imagination?” 2026, arXiv:2603.16666.
- [2] N. Lambert, B. Amos, O. Yadan, and R. Calandra, “Objective mismatch in model-based reinforcement learning,” in *Proc. 2nd Conf. Learn. Dyn. Control (LADC)*, ser. Proc. Mach. Learn. Res., vol. 120, 2020, pp. 761–770.
- [3] X. Fu, G. Yang, P. Agrawal, and T. Jaakkola, “Learning task informed abstractions,” in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 139, 2021, pp. 3480–3491.
- [4] Nilaksh, S. Jha, A. Zholus, and S. Chandar, “Reconstruction or semantics? What makes a latent space useful for robotic world models,” 2026, arXiv:2605.06388.
- [5] Y. Yu *et al.*, “How should world models be evaluated for embodied decision-making? A decision-making-centric position,” 2026, arXiv:2606.15032.
- [6] N. Karaev, Y. Makarov, J. Wang, N. Neverova, A. Vedaldi, and C. Rupprecht, “CoTracker3: Simpler and better point tracking by pseudo-labelling real videos,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 6013–6022.
- [7] M. Oquab *et al.*, “DINOv2: Learning robust visual features without supervision,” *Trans. Mach. Learn. Res.*, 2024. [Online]. Available: <https://openreview.net/forum?id=a68SUt6zFt>
- [8] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proc. Robot. Sci. Syst. (RSS)*, 2023.
- [9] A. Brohan *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. 7th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 2165–2183.
- [10] D. Driess *et al.*, “PaLM-E: An embodied multimodal language model,” in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 202, 2023, pp. 8469–8488.
- [11] Open X-Embodiment Collaboration, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 6892–6903.
- [12] D. Ghosh *et al.*, “Octo: An open-source generalist robot policy,” in *Proc. Robot. Sci. Syst. (RSS)*, 2024.
- [13] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proc. 8th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 2679–2713.
- [14] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proc. Robot. Sci. Syst. (RSS)*, 2023.
- [15] C. Chi *et al.*, “Diffusion Policy: Visuomotor policy learning via action diffusion,” *Int. J. Robot. Res.*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [16] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [17] Y. Zhang *et al.*, “ImageWAM: Do world action models really need video generation, or just image editing?” 2026, arXiv:2606.19531.
- [18] W. Zhang *et al.*, “DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, pp. 24 195–24 228.
- [19] W. Li *et al.*, “FoMoVLA: Bridging visual foresight and motion guidance for VLA models,” 2026, arXiv:2607.14739.
- [20] C. Wang *et al.*, “Track4Action: Distilling world-centric 3D tracker into vision-language-action policies,” 2026, arXiv:2608.03727.
- [21] L. Qiu *et al.*, “Making foresight actionable: Repurposing representation alignment in world action models,” 2026, arXiv:2606.12217.
- [22] F. Ma *et al.*, “GeoSem-WAM: Geometry- and semantic-aware world action models,” 2026, arXiv:2606.03188.
- [23] B. Li *et al.*, “EgoWAM: World action models beyond pixels with in-the-wild egocentric human data,” 2026, arXiv:2607.08436.
- [24] J. Guan *et al.*, “Point tracking improves world action models,” 2026, arXiv:2605.23856.
- [25] Y. Chen *et al.*, “FlowWAM: Optical flow as a unified action representation for world action models,” 2026, arXiv:2607.13017.
- [26] J. Guo *et al.*, “Unified 4D world action modeling from video priors with asynchronous denoising,” 2026, arXiv:2604.26694.
- [27] J. Zhang *et al.*, “Learning 4D geometric priors for inference-efficient world action models,” 2026, arXiv:2607.05468.
- [28] S. Yuan *et al.*, “DreamWAM: Beyond RGB future prediction for world action models,” 2026, arXiv:2608.04996.
- [29] H. Bao *et al.*, “VLMO: Unified vision-language pre-training with mixture-of-modality-experts,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 32 897–32 912.
- [30] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [31] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=XVjTT1nw5z>
- [32] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 44 776–44 791.
- [33] S. Fei *et al.*, “LIBERO-Plus: A progressive robustness benchmark for visual-language-action models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026, pp. 38 574–38 583.

- [34] T. Chen *et al.*, “RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026, to be published.
- [35] J. Chen *et al.*, “MV-WAM: Manifold-aware world action model with value augmentation,” 2026, arXiv:2606.21088.
- [36] K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 305, 2025, pp. 17–40.
- [37] L. Li *et al.*, “Causal world modeling for robot control,” in *Proc. Robot. Sci. Syst. (RSS)*, 2026.
- [38] H. Bi *et al.*, “Motus: A unified latent action world model,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026, pp. 35 101–35 113.
- [39] Q. Bu *et al.*, “UniVLA: Learning to act anywhere with task-centric latent actions,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [40] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [41] K. Pertsch *et al.*, “FAST: Efficient action tokenization for vision-language-action models,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [42] J. Cen *et al.*, “WorldVLA: Towards autoregressive action world model,” 2025, arXiv:2506.21539.
- [43] R. Zhao *et al.*, “SG-WAM: Self-guided world modeling in geometry-aware policy space,” 2026, arXiv:2608.01397.
- [44] M. Wang *et al.*, “ST-WAM: Semantic-temporal world action model for robust manipulation under visual distribution shifts,” 2026, arXiv:2607.28993.
- [45] L. Yang *et al.*, “4D-WAM: Infusing spatiotemporal awareness into world action models through trajectory fields,” 2026, arXiv:2608.08023.
- [46] Y. Yang *et al.*, “ABot-M0: VLA foundation model for robotic manipulation with action manifold learning,” 2026, arXiv:2602.11236.
- [47] S. Liu *et al.*, “RDT-1B: A diffusion foundation model for bimanual manipulation,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [48] J. Zhang, Y. Guo, Y. Hu, X. Chen, X. Zhu, and J. Chen, “UP-VLA: A unified understanding and prediction model for embodied agent,” in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, 2025, pp. 74 911–74 922.
- [49] Q. Shou *et al.*, “HALO: A unified vision-language-action model for embodied multimodal chain-of-thought reasoning,” in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026, to be published.
- [50] Y. Hu *et al.*, “BagelVLA: Enhancing long-horizon manipulation via interleaved vision-language-action generation,” in *Proc. Robot. Sci. Syst. (RSS)*, 2026, to be published.
- [51] J. Ye *et al.*, “StarVLA- α : Reducing complexity in vision-language-action systems,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2026, to be published.
- [52] Wan Team, “Wan: Open and advanced large-scale video generative models,” 2025, arXiv:2503.20314.